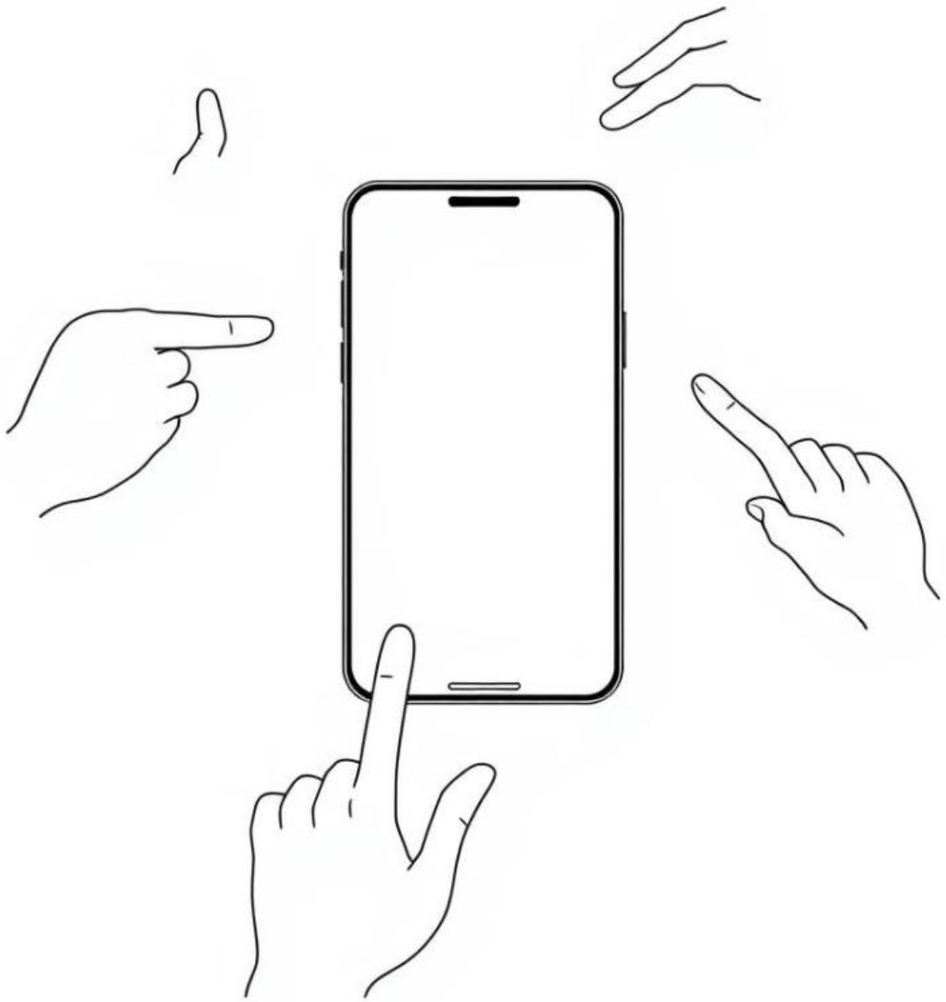


React Native Gesture Handler

React Native Gesture Handler (RNGH) is a library that provides native-driven gesture handling in React Native. This helps achieve more fluid gestures than the default React Native touch system, especially when building complex interactions like drag-and-drop, zoom, and multi-touch gestures.

Applications of React Native Gesture Handler

React Native Gesture Handler (RNGH) is widely used in React Native applications to improve gesture-based interactions

➢Enhancing User Interactions

- RNGH allows for smooth touch gestures, improving the overall UX/UI experience.
- Commonly used for interactions like tap, swipe, long press, and drag, making applications feel more natural and intuitive.

Example: Implementing a custom button with press and hold behavior instead of using default **TouchableOpacity**

➢Creating Custom Gesture-Based Navigation

- Works seamlessly with React Navigation, enabling gesture-based navigation transitions.
- Useful for swipe-back gestures, similar to iOS navigation.

➢Implementing Swipeable Components

- Swipe to delete (e.g., deleting messages like in WhatsApp).
- Swipeable cards (e.g., Tinder-like swiping interactions).

Applications of React Native Gesture Handler

React Native Gesture Handler (RNGH) is widely used in React Native applications to improve gesture-based interactions

➢ Drag and Drop Functionality

- RNGH makes it easy to implement drag-and-drop interfaces.
- Useful for applications that require reordering lists, moving UI elements, or dragging images.

➢ Zoom and Pinch Interactions

- Useful for image zooming, map interactions, and resizing UI elements.
- Works well with Reanimated for smooth animations.

➢ Gesture-Based Animations

- RNGH integrates with Reanimated to create highly interactive and performance-optimized animations.
- Enables smooth transitions, scaling, and physics-based effects.

Applications of React Native Gesture Handler

React Native Gesture Handler (RNGH) is widely used in React Native applications to improve gesture-based interactions

➢Interactive Bottom Sheets & Modals

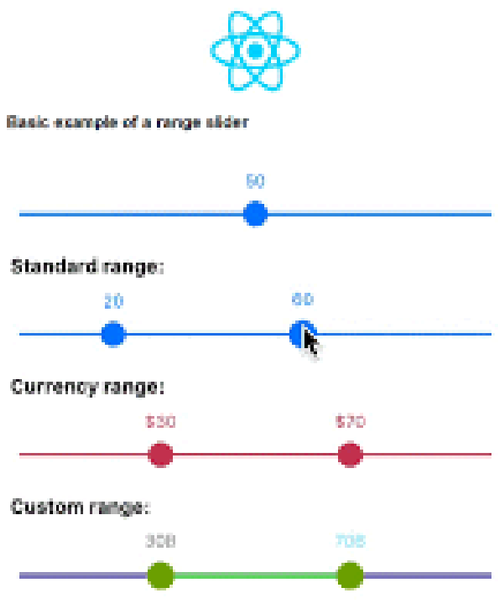
- RNGH is commonly used to build interactive bottom sheets and modals.
- Enables gestures for dragging up/down, dismissing by swiping, and snapping to predefined positions.

➢Game Controls & Interactions

- On-screen joystick controls.
- Swipe gestures for movement.
- Tap interactions for shooting or actions.

➢Custom Sliders and Controls

- Allows gesture-based custom sliders, improving the UX for media players, volume controls, and brightness settings.



Key Advantages of RNGH

1 **Native Processing**

Processes gestures on the native side, reducing latency and making interactions smoother.

2 **Built-in Gesture Types**

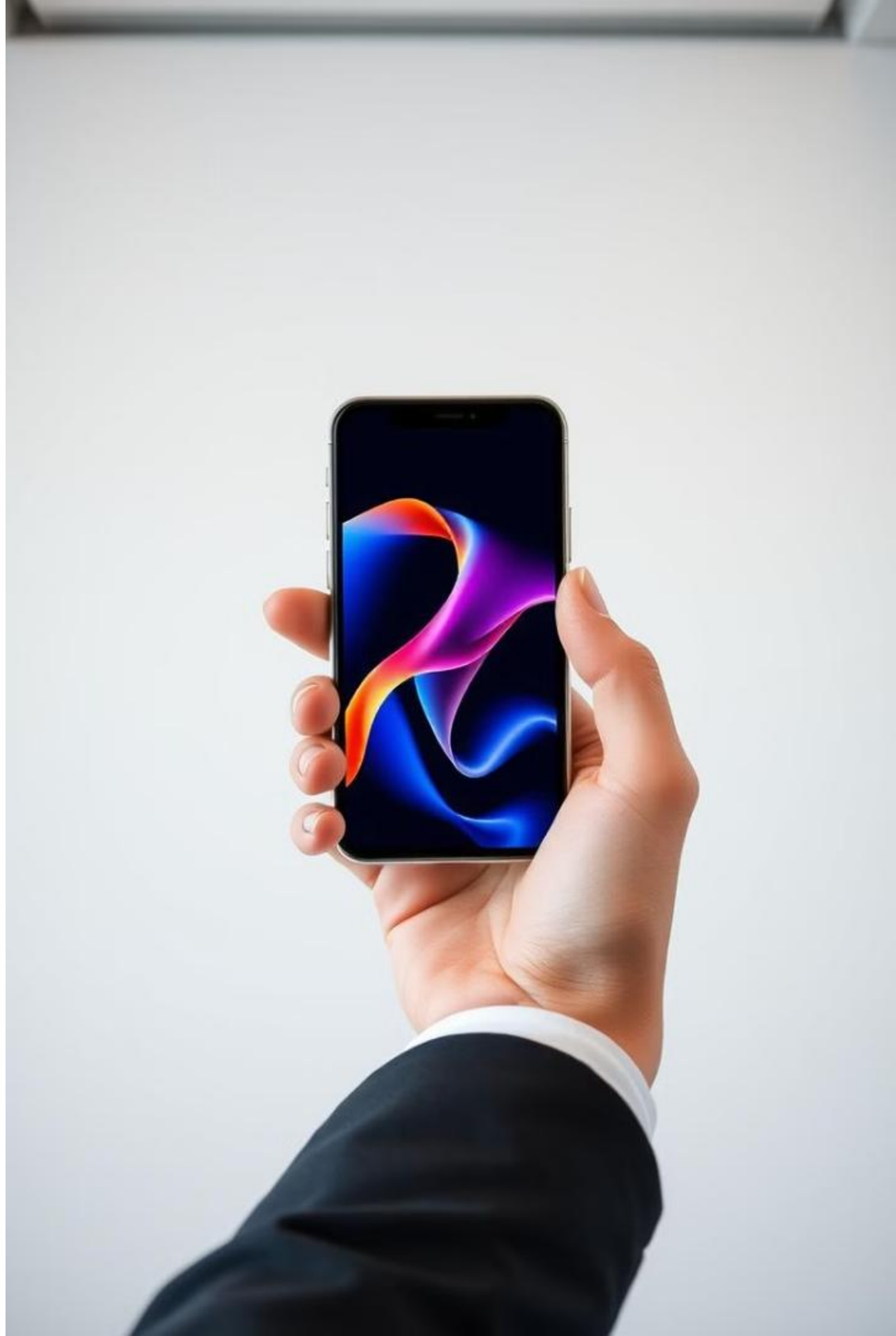
Provides built-in gesture types (Tap, Pan, Pinch, Rotation, Fling, etc.).

3 **Gesture Composition**

Allows gesture composition: using multiple gestures simultaneously or in sequence.

4 **Reanimated Integration**

Integrates well with React Native Reanimated to create smooth animations tied to gesture events.



Installation

- Install: *npm install react-native-gesture-handler*
- For react native CLI, Import react-native-gesture-handler in index.js or App.js to ensure proper functionality:

import 'react-native-gesture-handler';

- For **Expo users**, simply install the package:

expo install react-native-gesture-handler

TapGesture

Purpose

Detects tap interactions on a view. Can configure single tap, double tap, long press, etc.

Common Props / Callbacks

- onBegin, onEnd, onFailed, onActivated: Correspond to various gesture states (began, end, fail, success).
- numberOfTaps (iOS): e.g., set 2 to handle double tap.
- maxDeltaX, maxDeltaY: Maximum distance the finger can move and still be considered a tap.
- onActivated: Callback triggered when a tap is successfully recognized.

Example: TapGesture

```
import { View, StyleSheet } from 'react-native';
import { Gesture, GestureDetector } from 'react-native-gesture-handler';

export default function App() {
  const singleTap = Gesture.Tap()
    .maxDuration(250)
    .onStart(() => {
      console.log('Single tap!');
    });

  const doubleTap = Gesture.Tap()
    .maxDuration(250)
    .numberOfTaps(2)
    .onStart(() => {
      console.log('Double tap!');
    });
  return (
    <GestureDetector gesture={Gesture.Exclusive(doubleTap, singleTap)}>
      <View style={styles.box} />
    </GestureDetector>
  );
}
```

```
const styles = StyleSheet.create({
  box: {
    height: 120,
    width: 120,
    backgroundColor: '#b58df1',
    borderRadius: 20,
    marginBottom: 30,
  },
});
```

https://snack.expo.dev/@taitv/tap-gesture?platform=android

PanGesture

Purpose

Detects dragging (panning) across the screen. Useful for drag-and-drop, swipeable cards, etc.

Common Props / Callbacks

- onGestureEvent: Receives continuous updates about translationX, translationY, velocity, etc.
- activeOffsetX, activeOffsetY: Threshold that must be crossed in the horizontal/vertical axis for the gesture to be recognized.
- failOffsetX, failOffsetY: If movement doesn’t reach these thresholds, the gesture fails.
- minDist: Minimum distance before the gesture starts.
- minPointers, maxPointers: Minimum/maximum number of fingers to trigger this gesture.

Example: PanGesture

```
import { StyleSheet } from 'react-native';
import { Gesture, GestureDetector } from 'react-native-gesture-handler';
import Animated, {
  useSharedValue,
  withTiming,
  useAnimatedStyle,
} from 'react-native-reanimated';

const END_POSITION = 200;

export default function App() {
  const onLeft = useSharedValue(true);
  const position = useSharedValue(0);

  const panGesture = Gesture.Pan()
    .onUpdate((e) => {
      if (onLeft.value) {
        position.value = e.translationX;
      } else {
        position.value = END_POSITION + e.translationX;
      }
    })
    .onEnd((e) => {
      if (position.value > END_POSITION / 2) {
        position.value = withTiming(END_POSITION, { duration: 100 });
        onLeft.value = false;
      } else {
        position.value = withTiming(0, { duration: 100 });
        onLeft.value = true;
      }
    });

  const animatedStyle = useAnimatedStyle(() => ({
    transform: [{ translateX: position.value }],
  }));

  return (
    <GestureDetector gesture={panGesture}>
      <Animated.View style={[styles.box, animatedStyle]} />
    </GestureDetector>
  );
}

const styles = StyleSheet.create({
  box: {
    height: 120,
    width: 120,
    backgroundColor: '#b58df1',
    borderRadius: 20,
    marginBottom: 30,
  },
});
```

https://snack.expo.dev/@taitv/pan-gesture

PinchGesture

Purpose

Recognizes pinch-to-zoom gestures with two fingers. Outputs a scale value (1 is neutral, >1 zoom out, <1 zoom in).

Common Props / Callbacks

- onGestureEvent: Provides scale, focalX, focalY, velocity.
- minPointers: Usually set to 2 to detect pinch.
- enableTrackpadTwoFingerGesture (iOS): Allows two-finger gestures on a trackpad.

Example: PinchGesture

```
import React from 'react';
import { Dimensions, StyleSheet } from 'react-native';
import {
  Gesture,
  GestureDetector,
  GestureHandlerRootView,
} from 'react-native-gesture-handler';
import Animated, {
  useSharedValue,
  useAnimatedStyle,
} from 'react-native-reanimated';

const { width, height } = Dimensions.get('screen');

function clamp(val, min, max) {
  return Math.min(Math.max(val, min), max);
}

export default function App() {
  const scale = useSharedValue(1);
  const startScale = useSharedValue(0);

  const pinch = Gesture.Pinch()
    .onStart(() => {
      startScale.value = scale.value;
    })
    .onUpdate((event) => {
      scale.value = clamp(
        startScale.value * event.scale,
        0.5,
        Math.min(width / 100, height / 100)
      );
    })
    .runOnJS(true);
```

https://snack.expo.dev/@taitv/pinch-gesture

```
  const boxAnimatedStyles = useAnimatedStyle(() => ({
    transform: [{ scale: scale.value }],
  }));

  return (
    <GestureHandlerRootView style={styles.container}>
      <GestureDetector gesture={pinch}>
        <Animated.View style={[styles.box, boxAnimatedStyles]}>
        </Animated.View>
      </GestureDetector>
    </GestureHandlerRootView>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    alignItems: 'center',
    justifyContent: 'center',
  },
  box: {
    width: 100,
    height: 100,
    borderRadius: 20,
    backgroundColor: '#b58df1',
  },
  dot: {
    width: 24,
    height: 24,
    borderRadius: 12,
    backgroundColor: '#ccc',
    position: 'absolute',
    left: '50%'
```

RotationGesture

Purpose

Detects two-finger rotation. Reports rotation in radians.

Common Props / Callbacks

- onGestureEvent: Provides rotation, anchorX, anchorY, velocity.
- minPointers, maxPointers: Minimum and maximum number of fingers.

Example: RotationGesture

```
import React from 'react';
import { StyleSheet } from 'react-native';
import {
  Gesture,
  GestureDetector,
  GestureHandlerRootView,
} from 'react-native-gesture-handler';
import Animated, {
  useSharedValue,
  useAnimatedStyle,
} from 'react-native-reanimated';

export default function App() {
  const angle = useSharedValue(0);
  const startAngle = useSharedValue(0);

  const rotation = Gesture.Rotation()
    .onStart(() => {
      startAngle.value = angle.value;
    })
    .onUpdate((event) => {
      angle.value = startAngle.value + event.rotation;
    });

  const boxAnimatedStyles = useAnimatedStyle(() => ({
    transform: [{ rotate: `${angle.value}rad` }],
  }));

  return (
    <GestureHandlerRootView style={styles.container}>
      <GestureDetector gesture={rotation}>
        <Animated.View style={[styles.box, boxAnimatedStyles]}><
      </GestureDetector>
    </GestureHandlerRootView>
  );
```

```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    alignItems: 'center',
    justifyContent: 'center',
  },
  box: {
    width: 100,
    height: 100,
    borderRadius: 20,
    backgroundColor: '#b58df1',
  },
  dot: {
    width: 24,
    height: 24,
    borderRadius: 12,
    backgroundColor: '#ccc',
    position: 'absolute',
    left: '50%',
    top: '50%',
  },
});
```

https://snack.expo.dev/@taitv/rotation-gesture

FlingGesture

Purpose

Detects a rapid flick in a specific direction (UP, DOWN, LEFT, RIGHT).

Use Cases

Swipe-to-dismiss actions, quick navigation gestures.

Example: FlingGesture

```
import React from 'react';
import {
  Directions,
  Gesture,
  GestureDetector,
  GestureHandlerRootView,
} from 'react-native-gesture-handler';
import { Dimensions, StyleSheet } from 'react-native';
import Animated, {
  withTiming,
  useSharedValue,
  useAnimatedStyle,
} from 'react-native-reanimated';

const { width } = Dimensions.get('screen');

function clamp(val, min, max) {
  return Math.min(Math.max(val, min), max);
}

export default function App() {
  const translateX = useSharedValue(0);
  const startTranslateX = useSharedValue(0);

  const fling = Gesture.Fling()
    .direction(Directions.LEFT | Directions.RIGHT)
    .onBegin((event) => {
      startTranslateX.value = event.x;
    })
    .onStart((event) => {
      translateX.value = withTiming(
        clamp(
          translateX.value + event.x - startTranslateX.value,
          width / -2 + 50,
          width / 2 - 50
        ),
        { duration: 200 }
      );
    })
    .runOnJS(true);

  const boxAnimatedStyles = useAnimatedStyle(() => ({
    transform: [{ translateX: translateX.value }],
  }));

  return (
    <GestureHandlerRootView style={styles.container}>
      <GestureDetector gesture={fling}>
        <Animated.View style={[styles.box, boxAnimatedStyles]}><
      </GestureDetector>
    </GestureHandlerRootView>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    alignItems: 'center',
    justifyContent: 'center',
  },
  box: {
    width: 100,
    height: 100,
    borderRadius: 20,
    backgroundColor: '#b58df1',
    cursor: 'grab',
  },
});
```

https://snack.expo.dev/@taitv/fling-gesture

LongPressGesture

1 **Purpose**

Detects a prolonged press on a view.

2 **Use Cases**

Contextual menus, drag-and-drop interactions, item highlighting.

Example: LongPressGesture

```
import React from 'react';
import {
  Gesture,
  GestureDetector,
  GestureHandlerRootView,
} from 'react-native-gesture-handler';
import { Easing, StyleSheet } from 'react-native';
import Animated, {
  interpolateColor,
  useAnimatedStyle,
  useSharedValue,
  withTiming,
} from 'react-native-reanimated';

const COLORS = ['#b58df1', '#fa7f7c', '#ffe780', '#82cab2'];

export default function App() {
  const colorIndex = useSharedValue(0);
  const scale = useSharedValue(1);

  const longPress = Gesture.LongPress()
    .onBegin(() => {
      scale.value = withTiming(1.2, {
        duration: 500,
        easing: Easing.bezier(0.31, 0.04, 0.03, 1.04),
      });
    })
    .onStart(() => {
      colorIndex.value = withTiming(
        (colorIndex.value + 1) % (COLORS.length + 1),
        { duration: 200 },
        () => {
          if (colorIndex.value === COLORS.length) {
            colorIndex.value = 0;
          }
        }
      );
    })
```

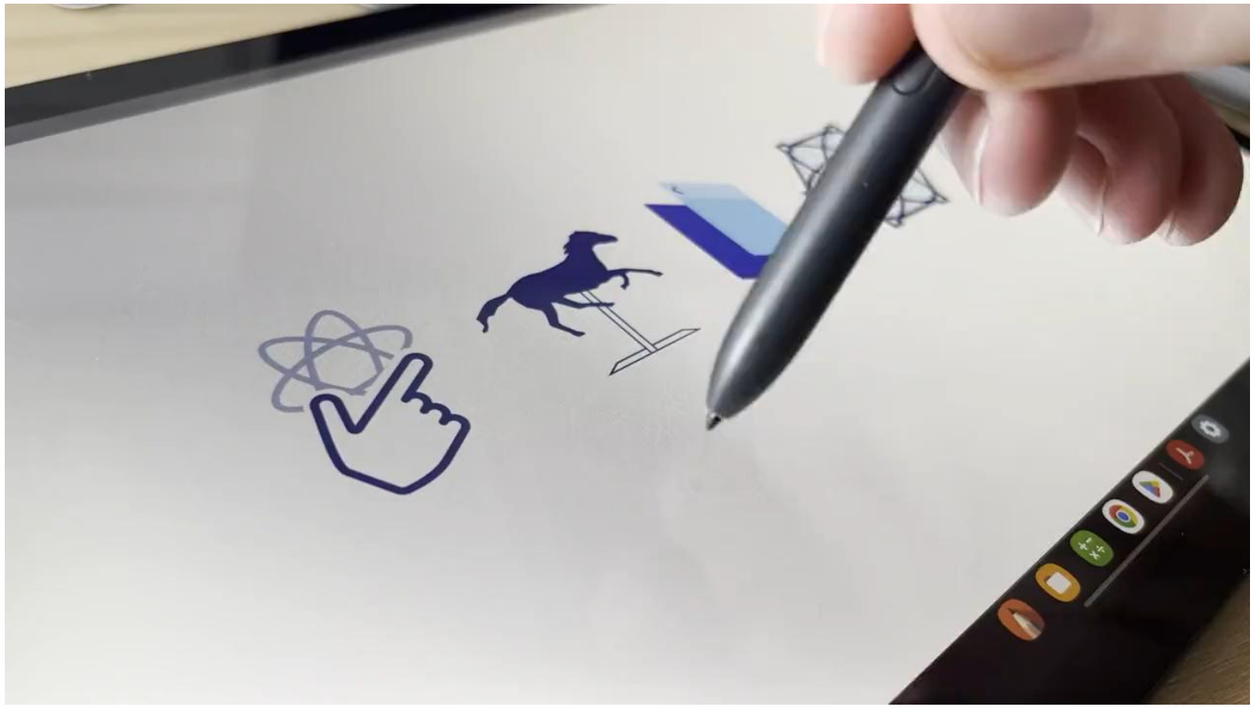
```
  const animatedStyle = useAnimatedStyle(() => ({
    backgroundColor: interpolateColor(
      colorIndex.value,
      [...COLORS.map((_, i) => i), COLORS.length],
      [...COLORS, COLORS[0]]
    ),
    transform: [{ scale: scale.value }],
  }));

  return (
    <GestureHandlerRootView style={styles.container}>
      <GestureDetector gesture={longPress}>
        <Animated.View style={[styles.box, animatedStyle]}>
      </GestureDetector>
    </GestureHandlerRootView>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    alignItems: 'center',
    justifyContent: 'center',
  },
  box: {
    width: 100,
    height: 100,
    borderRadius: 20,
    cursor: 'pointer',
  },
});
```

https://snack.expo.dev/@taitv/longpressgesture

HoverGesture



Purpose

Detects when a pointer (mouse or stylus) hovers over a component.

Use Cases

Interactive elements, tooltips, hover-based effects.

Example: HoverGesture

```
import React from 'react';
import {
  Gesture,
  GestureDetector,
  GestureHandlerRootView,
} from 'react-native-gesture-handler';
import { StyleSheet } from 'react-native';
import Animated, {
  Easing,
  interpolateColor,
  useAnimatedStyle,
  useSharedValue,
  withTiming,
} from 'react-native-reanimated';

const EASING = Easing.bezier(1, -1, 0.3, 1.43);

export default function App() {
  const translateX = useSharedValue(0);
  const translateY = useSharedValue(0);

  const progress = useSharedValue(0);

  const startX = useSharedValue(0);
  const startY = useSharedValue(0);

  const hover = Gesture.Hover()
    .onStart((event) => {
      startX.value = event.x;
      startY.value = event.y;
    })
    .onUpdate((event) => {
      translateX.value = (event.x - startX.value) * 0.3;
      translateY.value = (event.y - startY.value) * 0.3;

      const distance = Math.sqrt(Math.pow(translateX.value, 2) + Math.pow(translateY.value, 2));

      progress.value = distance / 35;
    })
    .onEnd(() => {
      translateX.value = withTiming(0, {
        duration: 400,
        easing: EASING,
      });
      translateY.value = withTiming(0, {
        duration: 400,
        easing: EASING,
      });
      progress.value = withTiming(0, {
        duration: 400,
        easing: EASING,
      });
    });

  const boxAnimatedStyle = useAnimatedStyle(() => ({
    transform: [
      { translateX: translateX.value },
      { translateY: translateY.value },
    ],
    backgroundColor: interpolateColor(
      progress.value,
      [0, 1],
      ['#b58df1', '#fa7f7c']
    )
  }));

  return (
    <GestureHandlerRootView style={styles.container}>
      <GestureDetector gesture={hover}>
        <Animated.View style={[styles.box, boxAnimatedStyle]}></Animated.View>
      </GestureDetector>
    </GestureHandlerRootView>
  );
}
```

https://snack.expo.dev/@taitv/longpressgesture

ForceTouchGesture: Pressure Sensitivity (only Ios)

Purpose

Detects the force applied during a touch.

Use Cases

3D Touch interactions, pressure-sensitive controls.

ForceTouchGesture: Pressure Sensitivity (only Ios)

```
import React from 'react';
import { View, StyleSheet } from 'react-native';
import { ForceTouchGestureHandler, State } from 'react-native-gesture-handler';
import Animated, { useSharedValue, useAnimatedStyle, interpolate } from 'react-native-reanimated';

export default function ForceTouchExample() {
  const force = useSharedValue(0);

  const animatedStyle = useAnimatedStyle(() => ({
    backgroundColor: `rgba(255, 0, 0, ${interpolate(force.value, [0, 1], [0.2, 1])})`,
  }));

  return (
    <ForceTouchGestureHandler
      minForce={0.1}
      onGestureEvent={event => {
        force.value = event.force;
      }}
      onHandlerStateChange={event => {
        if (event.nativeEvent.state === State.END) {
          force.value = 0; // Reset force on release
        }
      }}>
      <Animated.View style={[styles.box, animatedStyle]} />
    </ForceTouchGestureHandler>
  );
}

const styles = StyleSheet.create({
  box: {
    width: 150,
    height: 150,
    backgroundColor: 'red',
    alignSelf: 'center',
    marginTop: 50,
  },
});
```

Exercise: Using React Native Gesture Handler to Interact with Images

Let's create a React Native app that displays an image and allows users to interact with it using different types of gestures.
Each gesture will perform a specific action:

- Tap Gesture: When you tap on an image, the image will change randomly.
- Long Press Gesture: When you hold down an image for 1 second, a notification will appear.
- Pan Gesture: Allows users to drag the image across the screen.
- Pinch Gesture: Use two fingers to zoom in or out of an image.
- Rotation Gesture: Use two fingers to rotate an image.
- Fling Gesture: Quickly swipe to the right to fade the image out and disappear.